

LAB TASK 11

LOCAL VARIABLES AND DISPLAY

TASK 1: GENERATE SQUARE USING LOCAL VARIABLE

Code
<pre>[org 0x0100] jmp start generateSquare: push bp ; Push bp into stack and mov sp into bp so that both points to same memory location mov bp, sp sub sp, 2 Allocate space for 1 word (it creates local variable) mov ax, [bp + 4] ; Get the number into ax mul ax Mul ax with itself mov [bp - 2], ax ; Store the result in local variable mov ax, [bp - 2] mov sp, bp ; Load the result back to ax from local space and return pop bp ret 2 start: mov ax, 3 ; Mov 3 into ax register. push ax ; Push ax into stack call generateSquare ; Call the subroutine generate square mov ax, 0x4C00 int 0x21</pre>

TASK 2: SUM OF ARRAY USING SUBROUTINE

Code
<pre>[org 0x0100] jmp start arr: dw 2, 4, 6, 8, 10 sumArray: push bp mov bp, sp sub sp, 2 mov si, [bp + 4] ; Pointer to the array mov cx, [bp + 6] ; Number of elements xor ax, ax ; Clear ax to use as sum sum_loop: add ax, [si] add si, 2 loop sum_loop mov [bp - 2], ax mov ax, [bp - 2] ; Load sum into ax for return mov sp, bp pop bp ret 4 start: mov ax, arr</pre>

```

mov cx, 5
push cx
push ax
call sumArray

```

TASK 3: DISPLAY '#' IN BRIGHT WHITE ON BLUE

Code
<pre> [org 0x0100] jmp start calculateOffset: push bp mov bp, sp sub sp, 2 mov ax, [bp + 4] ; Get the row number mov bx, [bp + 6] ; Get the column number mov cx, 80 ; Number of columns per row mul cx ; AX = row * 80 add ax, bx ; AX = (row * 80) + column mul 2 ; Each character cell is 2 bytes mov [bp - 2], ax ; Store the result mov ax, [bp - 2] ; Load the result into ax for return mov sp, bp pop bp ret 4 start: mov ax, 0xB800 mov es, ax ; Set ES to point to video memory segment mov bx, 8 mov cx, 5 push cx ; Push column number push bx ; Push row number call calculateOffset ; AX now contains the offset in video memory mov di, ax ; Move offset to DI mov al, '#' ; Character to display mov ah, 0x1E ; Attribute byte (Bright white on blue) mov [es : di], ax ; Write character and attribute to video memory mov ah, 0x0 int 0x16 ; Wait for a key press mov ax, 0x4C00 int 0x21 </pre>

TASK 4: CLEAR THE ROWS AND COL

Code
<pre> [org 0x0100] jmp start start: mov ax, 0xB800 mov es, ax </pre>

```

mov ax, 5 ; 5th row
mov dx, 80 ; for formula (Row * 80 + col) * 2
mul dx
shl ax, 1

mov di, ax

mov cx, 240

loop1:
    mov [es : di], 0x0720
    add di, 2
    loop loop1

mov ah, 0x0
int ax, 0x16

mov ax, 0x4c00
int 0x21

```

TASK 5:

Code

```

[org 0x0100]
jmp start
message: db 'Hello, World!'
length: dw 13

printAt:
    push bp
    mov bp, sp
    push es
    push ax
    push bx
    push cx
    push dx
    push si
    push di

    mov ax, 0xB800
    mov es, ax

    mov si, [bp + 10] ; String address
    mov cx, [bp + 8] ; length
    mov bx, [bp + 6] ; Row
    mov dx, [bp + 4] ; Column

    mov ax, bx
    mov bx, 80
    mul bx
    add ax, dx
    shl ax, 1

    mov di, ax

```

```

    mov ah, 0x0F
print_loop:
    mov al, [si]
    mov [es : di], ax
    add di, 2
    inc si
    loop print_loop

```

```

    pop di
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    pop es
    pop bp
    ret 8

```

```

start:
    mov ax, message

```

```

    push ax
    push word [length]
    push word 10
    push word 20
    call printAt

```

```

    mov ax, 0x0
    int 0x16

```

```

mov ax,0x4c00
int 0x21

```

TASK 6: REVERSE THE ARRAY

Code

```

[org 0x0100]
jmp start

data: dw 10, 20, 30, 40, 50, 60
count: dw 6

; reverseArray(address, count)
; Parameters:
; [bp+6] = array address
; [bp+4] = count (number of elements)
; Uses one local variable at [bp-2] for swapping
reverseArray:
    push bp
    mov bp, sp
    sub sp, 2      ; Create 1 local variable for temp storage

    push ax
    push bx
    push cx

```

```

push si
push di

; Get parameters
mov si, [bp + 6] ; SI = left pointer (array start)
mov cx, [bp + 4] ; CX = count

; Calculate right pointer
; right = array_start + (count - 1) * 2
mov ax, cx ; AX = count
dec ax ; AX = count - 1
shl ax, 1 ; AX = (count - 1) * 2
add ax, si ; AX = array_start + (count - 1) * 2
mov di, ax ; DI = right pointer

; Calculate number of swaps needed = count / 2
shr cx, 1 ; CX = count / 2
jz reverse_done ; If count <= 1, nothing to swap

reverse_loop:
; Swap elements at [SI] and [DI] using local variable
mov ax, [si] ; AX = left element
mov [bp - 2], ax ; Store in local variable (temp)

mov ax, [di] ; AX = right element
mov [si], ax ; Put right element in left position

mov ax, [bp - 2] ; AX = temp (original left element)
mov [di], ax ; Put left element in right position

; Update pointers
add si, 2 ; Move left pointer right
sub di, 2 ; Move right pointer left

loop reverse_loop ; Repeat for all swaps

reverse_done:
pop di
pop si
pop cx
pop bx
pop ax

mov sp, bp ; Destroy local variable
pop bp
ret 4 ; Clean up 2 parameters (4 bytes)

start:
; Test with array: [10, 20, 30, 40, 50, 60]
mov ax, data
push ax ; Parameter 1: array address
push word [count] ; Parameter 2: count
call reverseArray

```

; After reverseArray, data will be: [60, 50, 40, 30, 20, 10]

```
mov ax, 0x4C00
int 0x21
```

TASK 7: DISPLAY '=' IN HORIZONTAL LINE 50 TIMES

Code

```
[org 0x0100]
jmp start

start:
    mov ax, 0xB800
    mov es, ax      ; Set ES to point to video memory segment

    ; Calculate starting offset for row 10, column 5
    ; Offset = (Row * 80 + Col) * 2
    ; For Row=10, Col=5: (10 * 80 + 5) * 2 = (800 + 5) * 2 = 805 * 2 = 1610
    mov ax, 10      ; Row
    mov dx, 80
    mul dx           ; AX = 10 * 80 = 800
    add ax, 5        ; AX = 800 + 5 = 805 (add column)
                    shl ax, 1      ; AX = 805 * 2 = 1610 (multiply by 2)

    mov di, ax       ; DI = starting offset (1610)

    mov cx, 50       ; Number of characters to display
    mov ax, 0x0A3D    ; AH = 0x0A (Bright Green on Black), AL = 0x3D ('=')

draw_line:
    mov [es: di], ax  ; Write '=' character with attribute to video memory
    add di, 2         ; Move to next character position (2 bytes)
    loop draw_line    ; Repeat 50 times

    mov ah, 0x0
    int 0x16          ; Wait for key press

    mov ax, 0x4C00
    int 0x21
```

TASK 8: DISPLAYS '*' FROM LEFT TO RIGHT IN ROW 12

Code

```
[org 0x0100]
jmp start

; Delay subroutine
delay:
    push cx
    push dx

    mov cx, 0x02      ; Outer loop counter (high word)
    mov dx, 0xFFFF    ; Inner loop counter (low word)
```

```

delay_loop:
    loop delay_loop ; Inner loop
    dec cx
    jnz delay_loop ; Outer loop

    pop dx
    pop cx
    ret

start:
    mov ax, 0xB800
    mov es, ax ; Set ES to point to video memory segment

    ; Calculate starting offset for row 12, column 0
    ; Offset = (Row * 80 + Col) * 2
    ; For Row=12, Col=0: (12 * 80 + 0) * 2 = 960 * 2 = 1920
    mov ax, 12 ; Row
    mov dx, 80
    mul dx ; AX = 12 * 80 = 960
    shl ax, 1 ; AX = 960 * 2 = 1920 (multiply by 2)

    mov di, ax ; DI = starting offset (1920)
    mov cx, 80 ; Number of columns to traverse

move_star:
    ; Display '*' at current position
    mov ax, 0x0F2A ; AH = 0x0F (Bright White), AL = 0x2A (*)
    mov [es : di], ax ; Write '*' to video memory

    ; Delay
    call delay

    ; Clear current position (replace with space)
    mov ax, 0x0720 ; AH = 0x07 (Normal White), AL = 0x20 (space)
    mov [es : di], ax ; Erase '*' from current position

    ; Move to next position
    add di, 2 ; Move to next column (2 bytes per character)

    loop move_star ; Repeat for all 80 columns

    mov ah, 0x0
    int 0x16 ; Wait for key press

    mov ax, 0x4C00
    int 0x21

```